

But even so, surely at some point we must actually have an effect on the world and submit the `Charge` for processing by some external system. And aren't there other useful programs that necessitate side effects or mutation? How do we write such programs? As we work through this book, we'll discover how many programs that seem to necessitate side effects have some functional analogue. In other cases we'll find ways to structure code so that effects occur but aren't *observable*. (For example, we can mutate data that's declared locally in the body of some function if we ensure that it can't be referenced outside that function, or we can write to a file as long as no enclosing function can observe this occurring.)

1.2 Exactly what is a (pure) function?

We said earlier that FP means programming with pure functions, and a pure function is one that lacks side effects. In our discussion of the coffee shop example, we worked off an informal notion of side effects and purity. Here we'll formalize this notion, to pinpoint more precisely what it means to program functionally. This will also give us additional insight into one of the benefits of functional programming: pure functions are easier to reason about.

A function f with input type A and output type B (written in Scala as a single type: $A \Rightarrow B$, pronounced “ A to B ” or “ A arrow B ”) is a computation that relates every value a of type A to exactly one value b of type B such that b is determined solely by the value of a . Any changing state of an internal or external process is irrelevant to computing the result $f(a)$. For example, a function `intToString` having type `Int => String` will take every integer to a corresponding string. Furthermore, if it really is a *function*, it will do nothing else.

In other words, a function has no observable effect on the execution of the program other than to compute a result given its inputs; we say that it has no side effects. We sometimes qualify such functions as *pure* functions to make this more explicit, but this is somewhat redundant. Unless we state otherwise, we'll often use *function* to imply no side effects.²

You should be familiar with a lot of pure functions already. Consider the addition (+) function on integers. It takes two integer values and returns an integer value. For any two given integer values, it will *always return the same integer value*. Another example is the `length` function of a `String` in Java, Scala, and many other languages where strings can't be modified (are immutable). For any given string, the same length is always returned and nothing else occurs.

We can formalize this idea of pure functions using the concept of *referential transparency* (RT). This is a property of *expressions* in general and not just functions. For the purposes of our discussion, consider an expression to be any part of a program that can be evaluated to a result—anything that you could type into the Scala interpreter

² *Procedure* is often used to refer to some parameterized chunk of code that may have side effects.